The Categorical Patterns of the Dobbertin (1) ⇒ (2) Formalisation

A TikZ field-guide to `DobbertinLego/Categorical.lean` and its golfed mirror `G/Cat.lean`

Abstract

The Lean formalisation of the opening step (1) ⇒ (2) of Dobbertin's Theorem 1 is assembled from a handful of components — *Frobenius*, the *loop/trace*, the *endomorphism*, the *telescope*, and the characteristic-2 inputs. This note traces each component back to its most abstract categorical home, and pairs the picture with the exact Lean declarations that realise it (module `DobbertinLego.Categorical`, golfed mirror `G.Cat`). The upshot is a clean four-pattern reading: Frobenius is a *monoid action* (a functor on the delooping $B\mathbb{N}$); the loop is the *norm element* generated by the *natural-number-object recursor*; the telescope is multiplication by the *augmentation* $\varphi - 1$ in the endomorphism *ring* (the geometric series); and the absolute trace is the *categorical (evaluation/coevaluation) trace*, equal to Mathlib's `Algebra.trace`. Two facts — additivity of Frobenius (char 2) and its finite order (Fermat) — are the irreducible arithmetic inputs where the abstract scaffold meets the finite field.

1 The dictionary

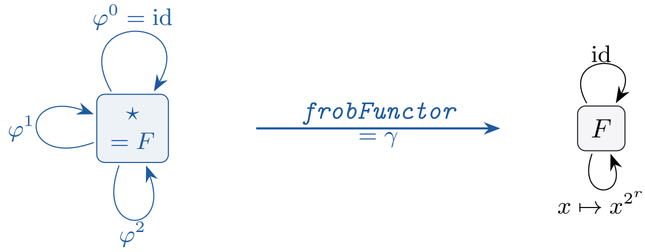
Every row is a theorem or definition in `DobbertinLego.Categorical` (golfed name in `G.Cat` in the last column).

component	**categorical foundation**	**Lean (Categorical)**	**golfed**
Frobenius φ (shift)	monoid action $\mathrm{Mult}\,\mathbb{N} \to \mathrm{End}$; a functor $B\mathbb{N} \rightsquigarrow$ **Type**	`frobAction`, `frobFunctor`	α, γ
loop / trace σ	norm element $\sum \varphi^i$, built by the **NNO** recursor on $\mathbb{N}$	`iterSum`, `iterSum_rec_unique`	σ, ρ
telescope	multiply by augmentation $\varphi - 1$ in the ring $\mathrm{End}\,A$ (geometric series)	`augmentation_mul_normElt`	ρ
finite order / bit	$\varphi^n = 1$ (cyclic Galois group) forces σ into the fixed subobject	`iterSum_fixed_of_orderly`	θ
absolute trace Tr	categorical trace (eval/coeval loop) = `Algebra.trace` $\mathbb{F}_2\,F$	`trace_eq_algebraMap_trace`	κ
characteristic 2	the *input* making φ an endomorphism of the *additive* group	`frob_add`	φ`a`

> **Reading.** The four coloured patterns (action, recursor, augmentation, trace) are *formal*: they hold for any endomorphism of any abelian group / any dualizable object. The two red rows are the only genuine finite-field inputs.

2 Pattern 1 — Frobenius is the shift: a monoid action / delooping

Additivity (`frob_add`) makes each iterate φ^r an endomorphism of the additive group F; composition (`frob_comp`) makes $r \mapsto \varphi^r$ a *monoid homomorphism* out of the free monoid on one generator, $\mathrm{Mult}\,\mathbb{N}$. Equivalently it is a *functor* out of the one-object category $B\mathbb{N} =$ `SingleObj (Multiplicative ℕ)`, i.e. an action of $\mathbb{N}$ on F.



$B\mathbb{N}$ = one object, morphisms = $\mathbb{N}$ (composition = addition) **Type** (or **AddCommGrp**)

```
noncomputable def frobAction : Multiplicative ℕ →* AddMonoid.End F where
  toFun r := frobEndo (Multiplicative.toAdd r)      -- r ↦ φ^r
  map_one' := ...    -- φ^0 = id
  map_mul' := ...    -- φ^(a+b) = φ^a ∘ φ^b

noncomputable def frobFunctor : SingleObj (Multiplicative ℕ) ⥤ Type _ :=
  SingleObj.functor frobTypeHom           -- BN ⥤ Type : the ℕ-action as a functor
```

A functor out of `SingleObj M` is *exactly* an action of the monoid M (Mathlib's `SingleObj.functor` takes a monoid hom $M \to \operatorname{End} X$). Finiteness of the field makes the action *factor through the cyclic quotient* $\operatorname{Mult}(\mathbb{N}/n\mathbb{N})$ — the Galois group $\operatorname{Gal}(\mathbb{F}_{2^n}/\mathbb{F}_2)$ generated by Frobenius — because $\varphi^n = 1$ (Pattern $1'$, `frobEndo_pow_card`).

3 Pattern 2 — the loop is the norm element, built by the NNO recursor

The loop $\sigma\,\varphi\,\ell = \sum_{i<\ell} \varphi^i$ is generated by the inductive pattern: it is the *unique* map $\mathbb{N} \to A$ with $g\,0 = 0$ and $g(\ell+1) = g\,\ell + \varphi^\ell x$. This is precisely the recursor of a natural-number object (the initial algebra of $X \mapsto \mathbf{1} + X$).

$$\begin{tikzcd}
\mathbf{1} \arrow[r, "0"] \arrow[dr, "z=0"'] & \mathbb{N} \arrow[r, "\mathrm{succ}"] \arrow[d, dashed, "g"] & \mathbb{N} \arrow[d, dashed, "g"] \\
& A \arrow[r, "a \mapsto a + \varphi^\ell x"'] & A
\end{tikzcd}$$

The unique fill-in g is $\ell \mapsto \sigma\,\varphi\,\ell\,x$; `iterSum_rec_unique` records exactly this universal property (Mathlib has no `NNO` class, so the recursor is stated directly).

```
theorem iterSum_rec_unique (φ : AddMonoid.End A) (x : A) (g : ℕ → A)
    (h0 : g 0 = 0) (hs : ∀ len, g (len + 1) = g len + (φ ^ len) x) :
    ∀ len, g len = iterSum φ len x
```

4 Pattern 3 — the telescope is the augmentation in the endomorphism ring

Both Artin–Schreier telescopes the paper uses are one ring identity. In the (noncommutative) endomorphism *ring* $\operatorname{End} A$, the norm element $N = \sum_{i<\ell} \varphi^i$ times the *augmentation* $\varphi - 1$ is the geometric series $\varphi^\ell - 1$:

$$(\varphi - 1) \cdot N \;=\; \varphi^\ell - 1 \qquad \text{(Mathlib's `mul_geom_sum`).}$$

Applying this endomorphism to x gives the pointwise telescope $\varphi(\sigma x) - \sigma x = \varphi^\ell x - x$; over $\mathbb{F}_2$ the subtraction becomes addition, giving the paper's "add the 2^k-th power to itself".

$$
\begin{tikzcd}
 & & \varphi^{\ell} x & & \\
x \arrow[r, "\varphi"] & \varphi x \arrow[r, "\varphi"] & \varphi^2 x \arrow[r, "\varphi"] & \cdots \arrow[r, "\varphi"] & \varphi^{\ell-1} x \arrow[ull, dashed, bend right]
\end{tikzcd}
$$

$$(\varphi-1)\big(\underbrace{x+\varphi x+\cdots+\varphi^{\ell-1}x}_{Nx=\sigma\,\varphi\,\ell\,x}\big)=\varphi^{\ell}x-x$$

```
def normElt (φ : AddMonoid.End A) (len : ℕ) : AddMonoid.End A := ∑ i ∈ range len, φ ^ i
theorem augmentation_mul_normElt (φ : AddMonoid.End A) (len : ℕ) :
    (φ - 1) * normElt φ len = φ ^ len - 1 := mul_geom_sum φ len
```

Finite-order corollary (the "bit"). When $\varphi^n = 1$ the right-hand side is 0, so N maps A into the fixed subgroup $\ker(\varphi - 1) = A^{\varphi}$. Over $\mathbb{F}_{2^n}$ this is $\mathbb{F}_2 = \{0, 1\}$, so $\mathrm{Tr}(x)$ is a bit (`iterSum_fixed_of_orderly` → `trace_isBit`; golfed θ→`Tb`).

5 Pattern 4 — the absolute trace is the categorical trace

The absolute trace $\mathrm{Tr}(x) = \sum_{i<n} x^{2^i}$ is not an ad-hoc sum: it is the *categorical trace* of the multiplication endomorphism m_x on the dualizable object F (a finite-dimensional $\mathbb{F}_2$-vector space) — the one "open a loop with coevaluation, insert m_x, close it with evaluation" move. Mathlib packages that value as `Algebra.trace`, and computes it over the prime field as the Frobenius sum, which is exactly the loop.

$$I \xrightarrow{\;coev\;} V \otimes V^* \xrightarrow{\;m_x \otimes \mathrm{id}\;} V \otimes V^* \xrightarrow{\;swap\;} V^* \otimes V \xrightarrow{\;eval\;} I$$

$$\text{categorical trace} \;=\; \mathrm{Tr}(x) \;=\; \textstyle\sum_{i<n} x^{2^i}$$

```
theorem trace_eq_algebraMap_trace (x : F) :
    letI := ZMod.algebra F 2
    trace (Module.finrank (ZMod 2) F) x
      = algebraMap (ZMod 2) F (Algebra.trace (ZMod 2) F x)
```

The identification is genuine, not aspirational: at full length $n = [F : \mathbb{F}_2]$ the paper's `trace` equals Mathlib's `Algebra.trace` pushed back along `algebraMap`, via `FiniteField.algebraMap_trace_eq_sum_pow`. *Additivity* of Tr is then just monoidality/linearity — no ad-hoc computation.

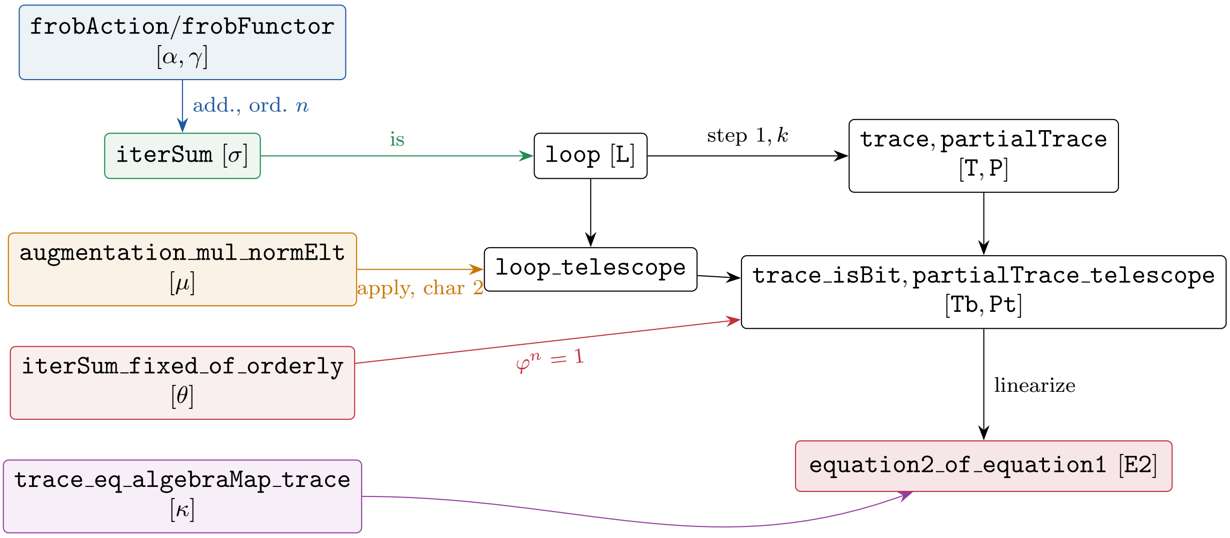
6 Formal vs. arithmetic: where the field enters

Formal (any endomorphism / any abelian group)	**Genuine finite-field input**
telescope $(\varphi - 1)N = \varphi^{\ell} - 1$ (μ)	char 2: φ additive (φ`a`, `frob_add`)
N lands in fixed points when $\varphi^n = 1$ (θ)	Fermat: $\varphi^n = 1$ (Φ`F`, `frobEndo_pow_card`)
σ as NNO recursor (ρ)	$A^{\varphi} = \mathbb{F}_2$ is a two-element set (the bit)
Frobenius action / delooping (α,γ)	periodicity mod n (φ`p`) from $x^{\lvert F\rvert} = x$

Category theory supplies the *shape* (a contraction/loop on a dualizable object with a distinguished endomorphism); the two red facts supply the *content* at the point of specialization to $\mathbb{F}_{2^n}$.

7 The whole picture

Each abstract pattern (left) specializes to a concrete brick over $\mathbb{F}_{2^n}$ (middle) and feeds the headline (1) ⇒ (2) (right). Golfed names in brackets.



Soundness. `DobbertinLego.Categorical` and the golfed `G` library build `sorry`-free; every headline (`trace_eq_algebraMap_trace`, `augmentation_mul_normElt`, `equation2_of_equation1` / `E2`, . . .) depends only on `propext`, `Classical.choice`, `Quot.sound`. The categorical layer *organises* the proof; the arithmetic is discharged by the two red inputs.